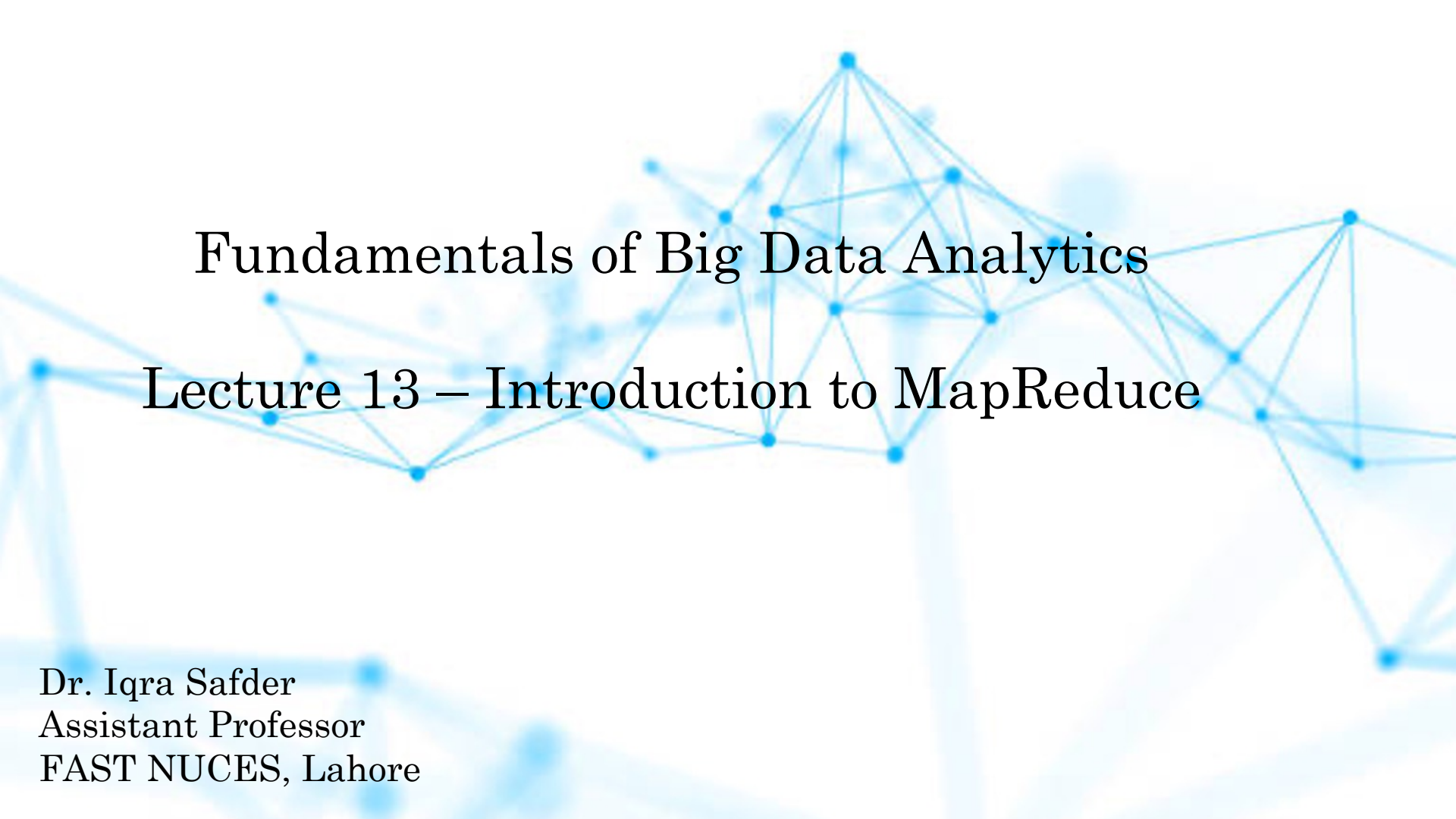


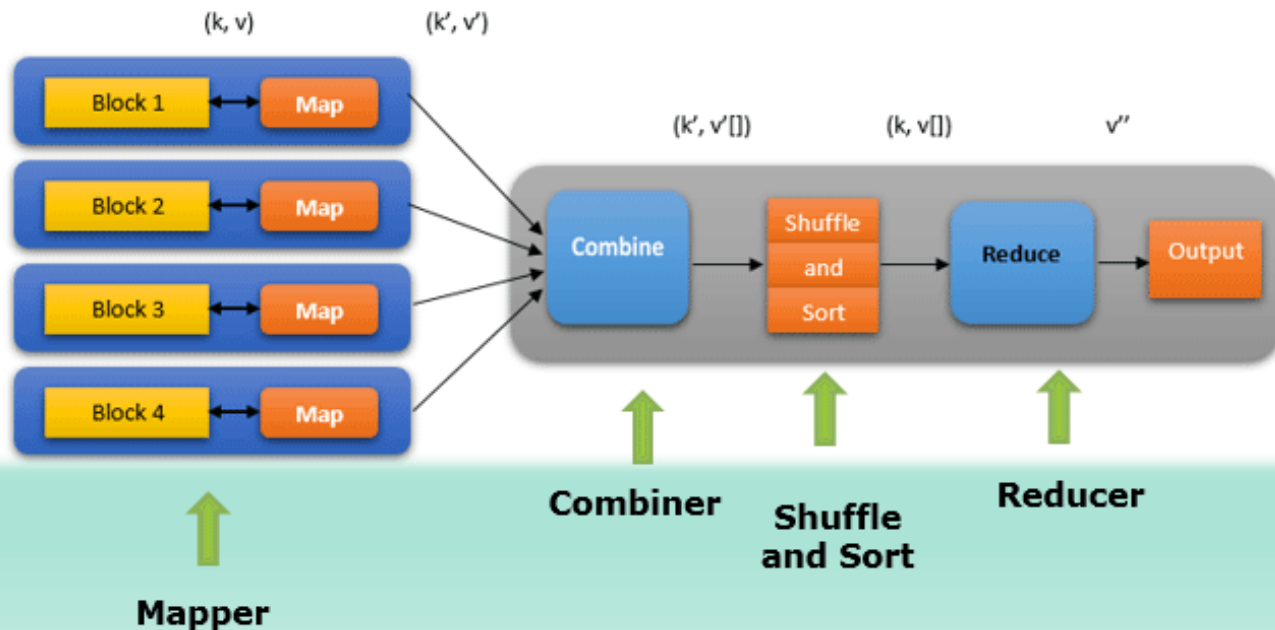


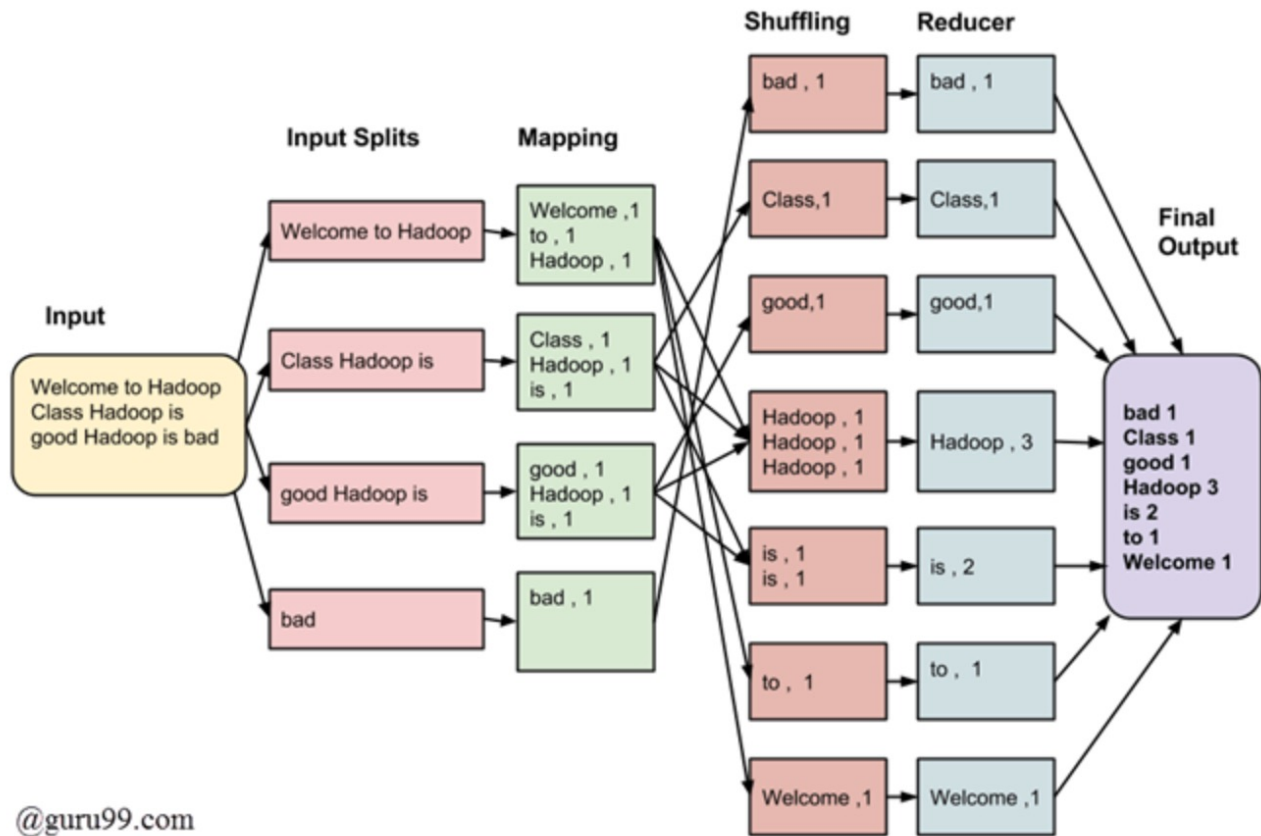
Fundamentals of Big Data Analytics

Lecture 13 – Introduction to MapReduce

Dr. Iqra Safder
Assistant Professor
FAST NUCES, Lahore

How MapReduce Works





MapReduce Architecture

Example: Vector Multiplication

Vector Multiplication

- Task: multiply 2 arrays of N numbers
 - A basic mathematical operation
 - Let's assume N is very large

Vector Multiplication

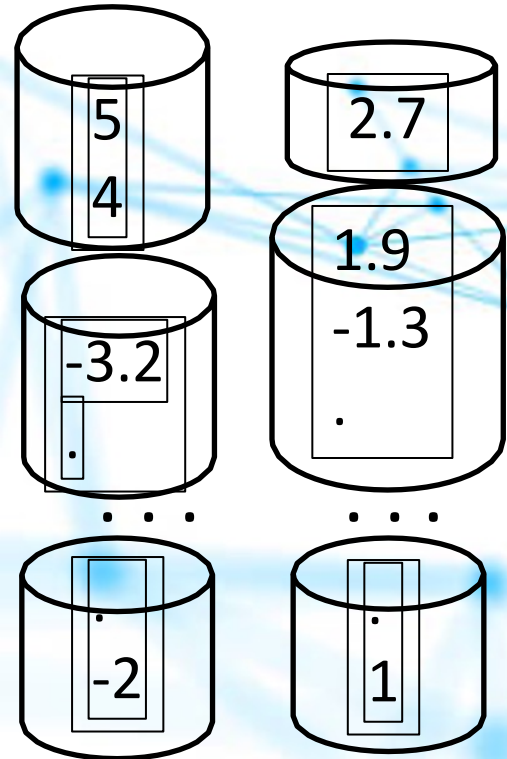
- Task: multiply 2 arrays of N numbers

A	X	B
5		2.7
4		1.9
3.2		1.3
.		.
.		.
.		.
-2		1



$$\begin{aligned} &= \\ &\quad (5 \times 2.7) \quad \# \text{ 1}^{\text{st}} \text{ of A \& B} \\ &+ (4 \times 1.9) \quad \# \text{ 2}^{\text{nd}} \text{ of A \& B} \\ &+ (3.2 \times 1.3) \quad \# \text{ 3}^{\text{rd}} \dots \\ &\dots \\ &\dots \\ &\dots \\ &+ (-2 \times 1) \quad \# \text{ Nth of A \& B} \end{aligned}$$

Vector Multiplication



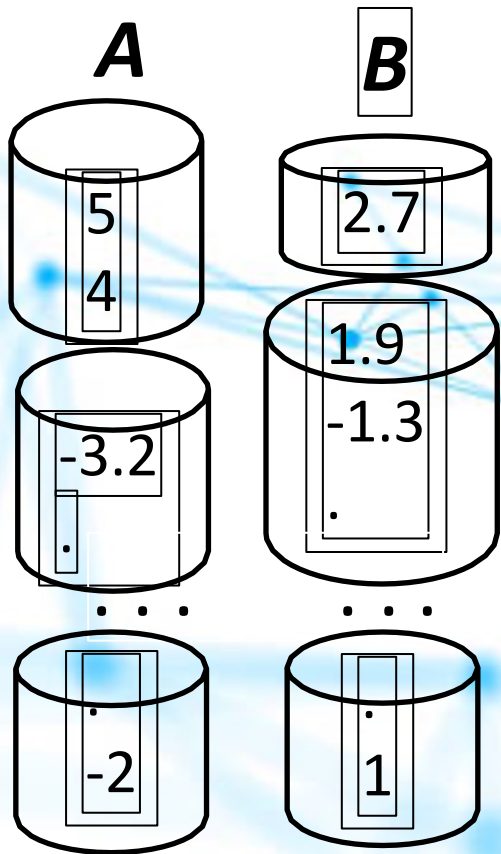
- Recall:
 - data partitioned in HDFS

Vector Multiplication

- Main design consideration:
need elements with same index together

*Let $\langle key, value \rangle =$
 $\langle index, number \rangle$*

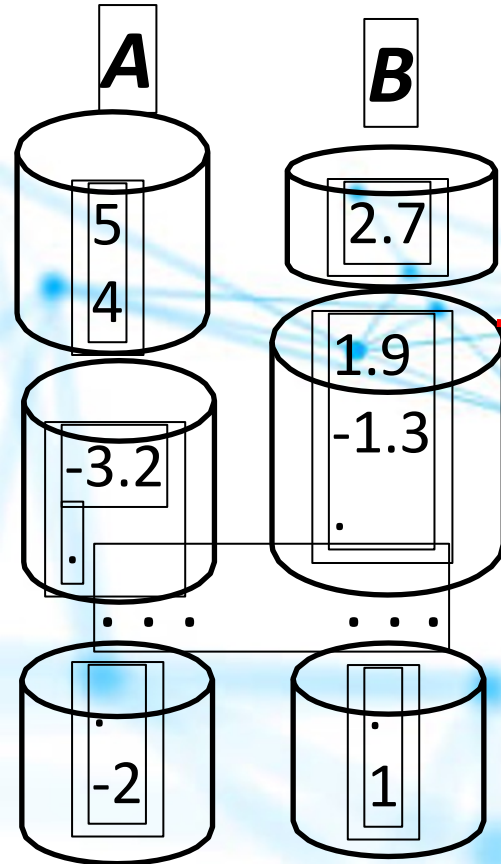
Vector Multiplication



- Problem: array partitions don't have an index



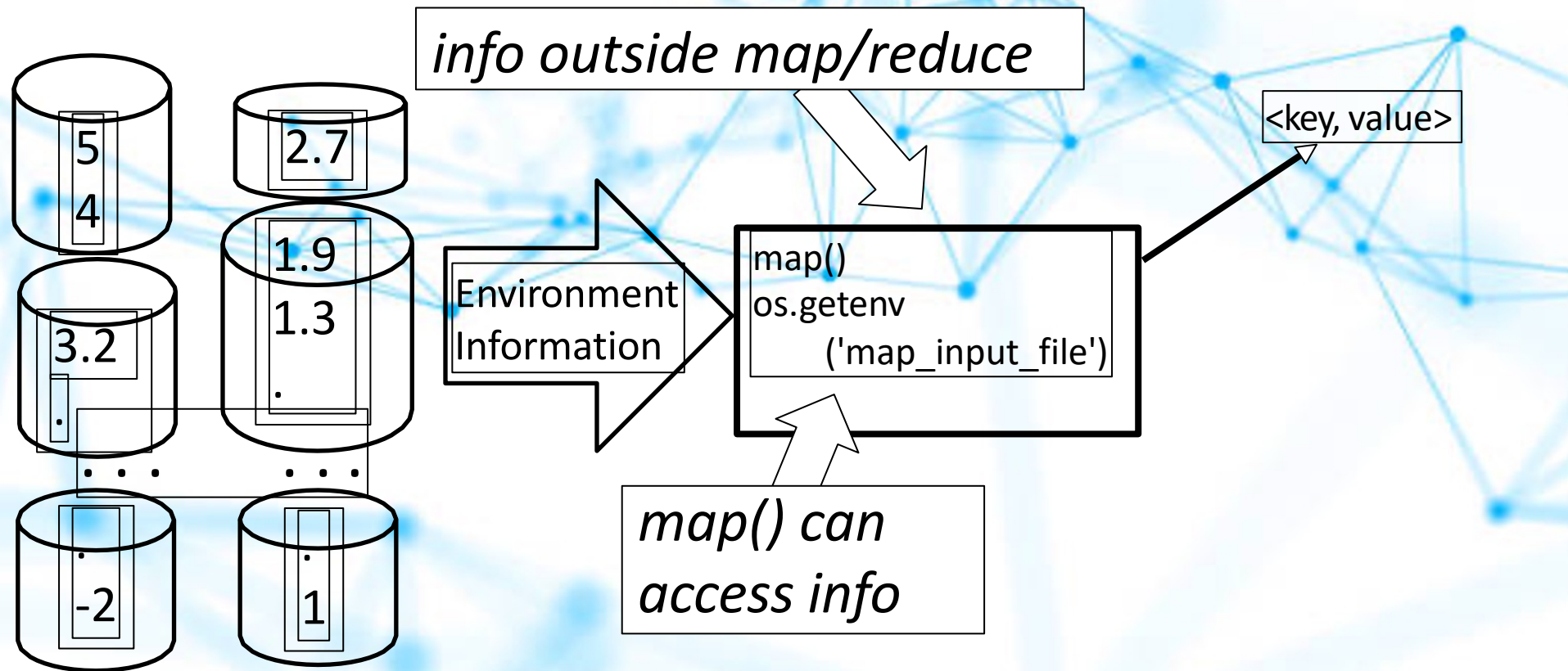
Vector Multiplication



*Hadoop makes environmental
Information available*

Environment
Information

Vector Multiplication



Vector Multiplication

A	B
1, 5 2, 4	1, 2.7
3, 3.2	2, 1.9 3, 1.3
...	...
N, -2	N, 1

- Let's assume:
← each line already has
<index, number>

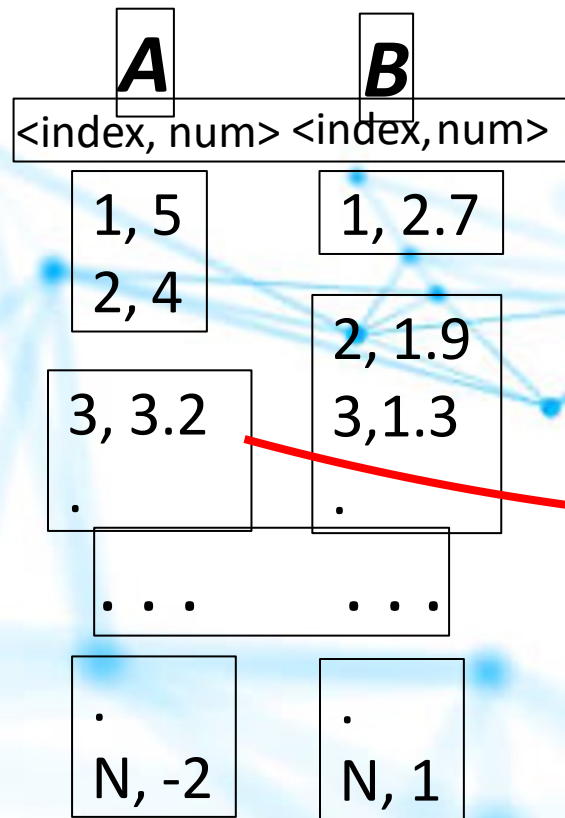
Vector Multiplication

A	B
1, 5 2, 4	1, 2.7
3, 3.2 .	2, 1.9 3, 1.3 .
...	...
. N, -2	. N, 1

- Let's assume:
← each line already has
<index, number>

*Note: mapper only needs to
pass data (identity function)*

Vector Multiplication



shuffle &
group indices

<index, num>

1, 5
1, 2.7
3, 1.3
3, 3.2
2, 1.9
2, 4
...

Vector Multiplication

1, 5
1, 2.7
3, 1.3
3, 3.2

2, 1.9
2, 4

...

What should
reducers do?

Vector Multiplication

A, B grouped

1, 5

1, 2.7

3, 1.3

3, 3.2

2, 1.9

2, 4

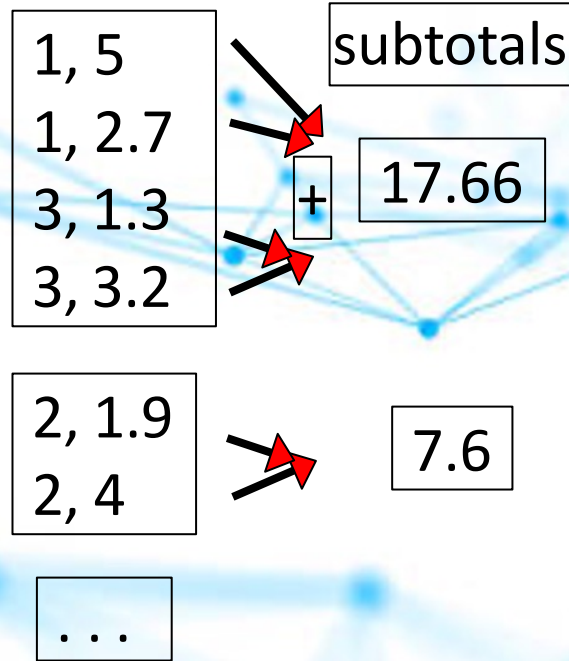
...

Reducer:

-get pairs of

<index, number>

Vector Multiplication



Reducer:

- get pairs of $\langle index, number \rangle$
- multiply & add

Vector Multiplication

1, 5
1, 2.7
3, 1.3
3, 3.2

subtotals

+ 17.66

2, 1.9
2, 4

7.6

...

Reducer:

-get *pairs of*
<index, number>
-multiply & add

*(Still need get total
sum, but should be
largely reduced)*



Computational Costs

In Vector Multiplication

Computational Costs

- For Vector Multiplication
 - *How many $\langle \text{index}, \text{number} \rangle$ are output from `map()`?*

Computational Costs

- For Vector Multiplication
 - *How many $\langle \text{index}, \text{number} \rangle$ are output from `map()`?*
 - *How many $\langle \text{index} \rangle$ groups have to be shuffled?*

Computational Costs

- How many <index, number> are output ?

A

1, 5
2, 4
3, -3.2
.
.
.
N, -2

B

1, 2.7
2, 1.9
3, -1.3
.
.
.
N, 1

For: 2 Vectors with
 N indices each

Then:

$2N$ <index, number>
are output from map()

Computational Costs

- How many <index> groups have to be shuffled?

A,B grouped

<index, num>

1, 5

1, 2.7

3, -1.3

3, -3.2

2, 1.9

2, 4

...

For: $2N$ indices and
 N pairs

Then:

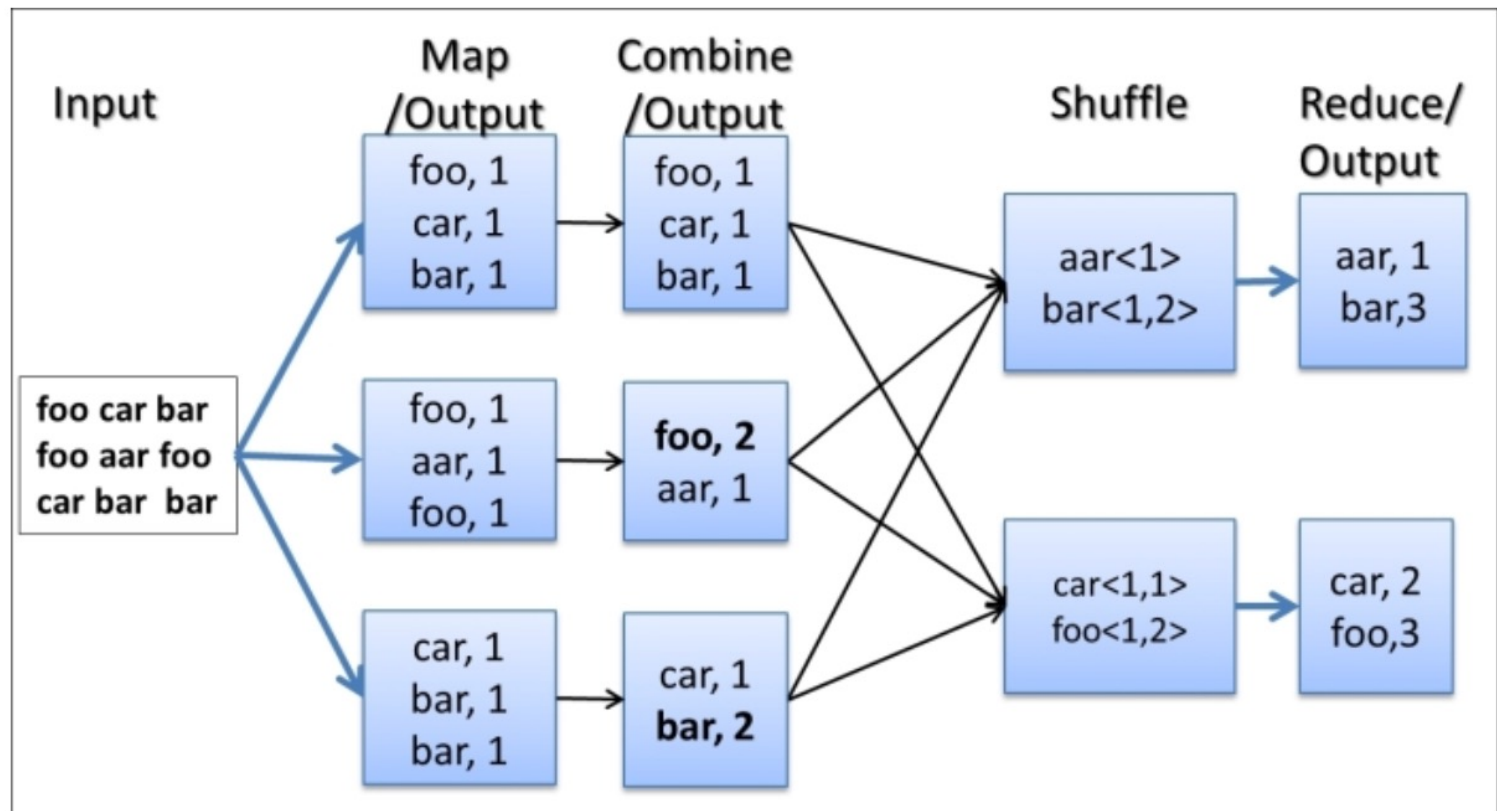
N groups are shuffled to
reducers

More groups more shuffling.

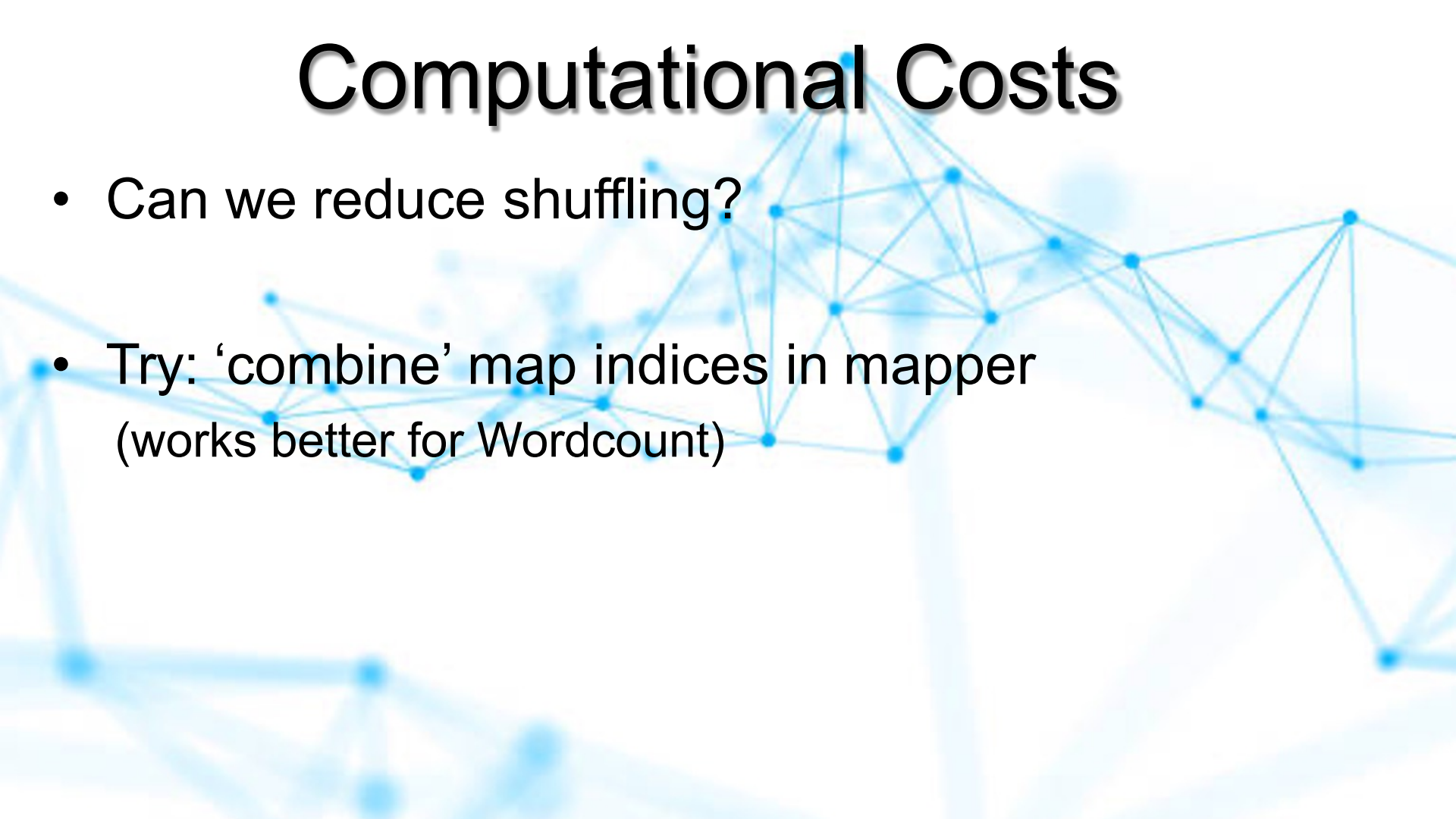
Computational Costs

- Can we reduce shuffling?



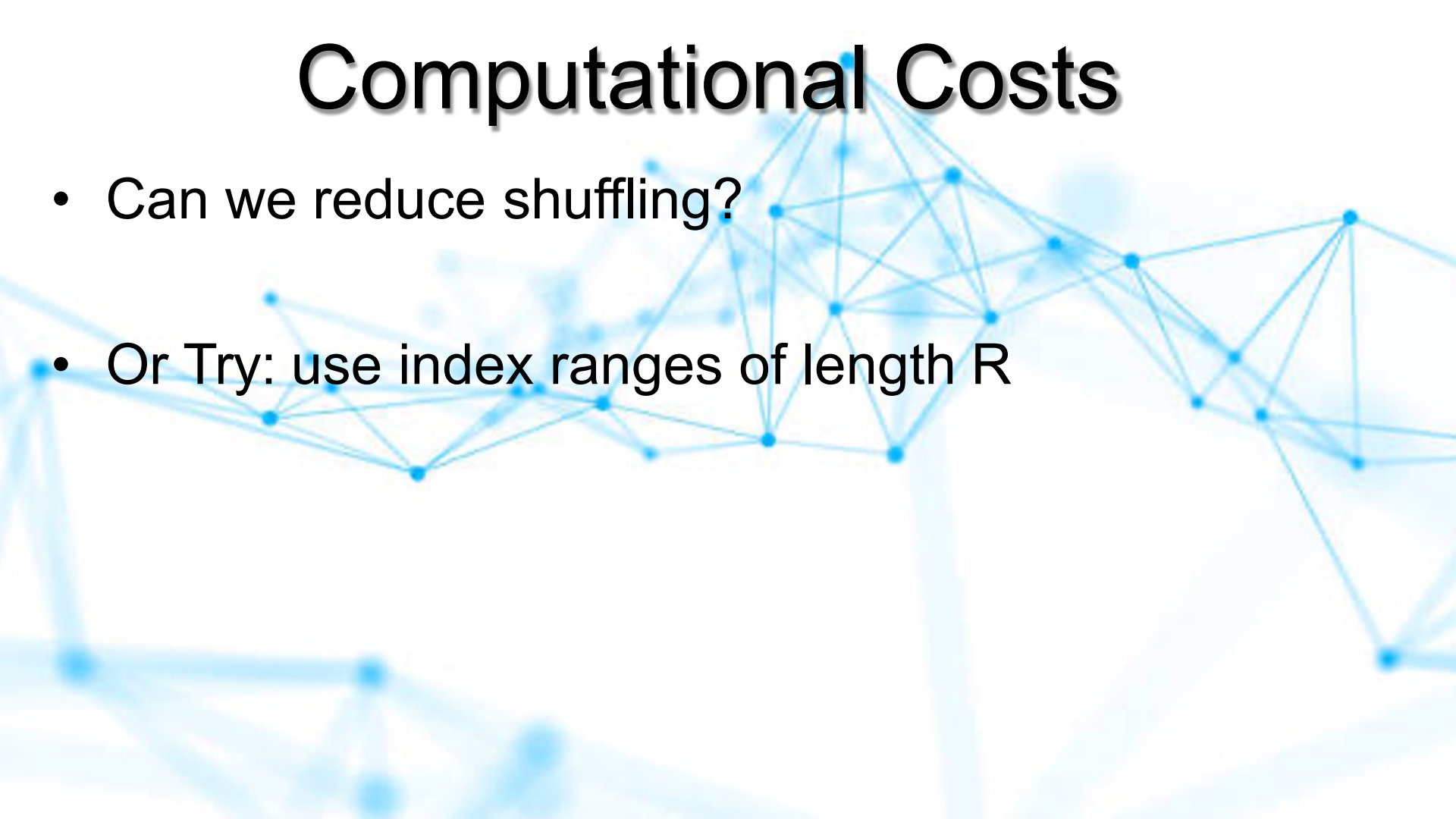


Computational Costs



- Can we reduce shuffling?
- Try: 'combine' map indices in mapper
(works better for Wordcount)

Computational Costs



- Can we reduce shuffling?
- Or Try: use index ranges of length R

Computational Costs

- Index Ranges: let $R=10$ & bin the array indices

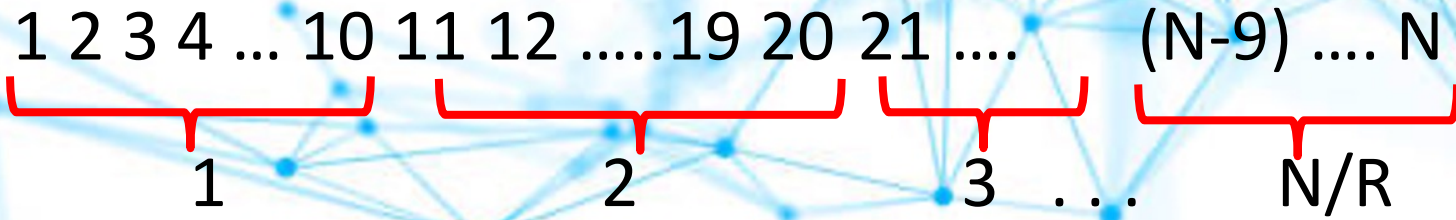
1 2 3 4 ... 10 11 1219 20 21 (N-9) ... N N keys

Computational Costs

- Index Ranges: let $R=10$ & bin the array indices

1 2 3 4 ... 10 11 12 19 20 21 (N-9) ... N

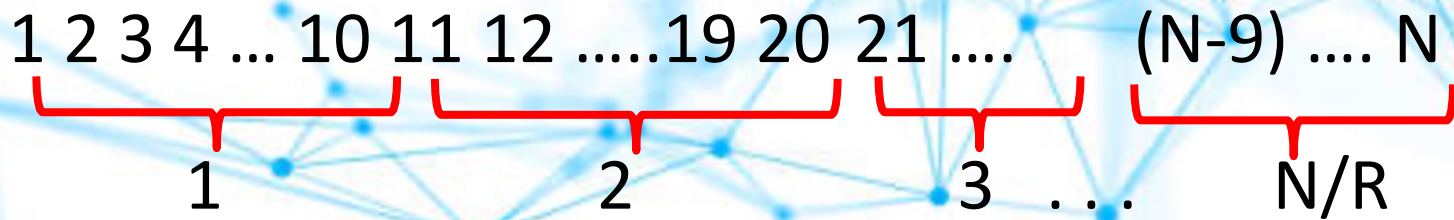
1 2 3 ... N/R



place
in
bins

Computational Costs

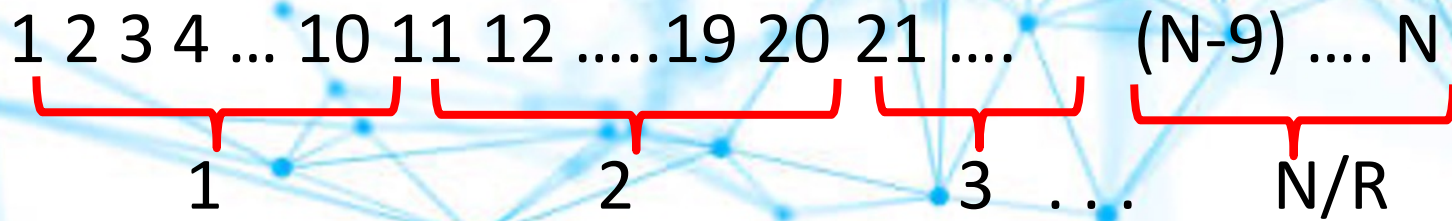
- For example, let $R=10$, and bin the array indices



N keys are now $N/R = N/10$ keys

Computational Costs

- For example, let $R=10$, and bin the array indices



N keys are now $N/R=N/10$ keys

$\langle \text{key}, \text{value} \rangle$ is now

$\langle \text{index bin}, \text{original-index number} \rangle$

Computational Costs

- Now shuffling costs depend on N/R groups

If: $R=1$

Then: $N/R=N$ groups (same as before)

If: $R>1$

Then: $N/R < N$ (less shuffling to do)

Computational Costs

If:

size of $(N/R) \uparrow$

Then:

shuffle costs $\uparrow$

Computational Costs

If:

size of $(N/R) \uparrow$

Then:

shuffle costs $\uparrow$

But:

reducer complexity $\downarrow$

Computational Costs

If:

size of $(N/R) \uparrow$

Then:

shuffle costs $\uparrow$

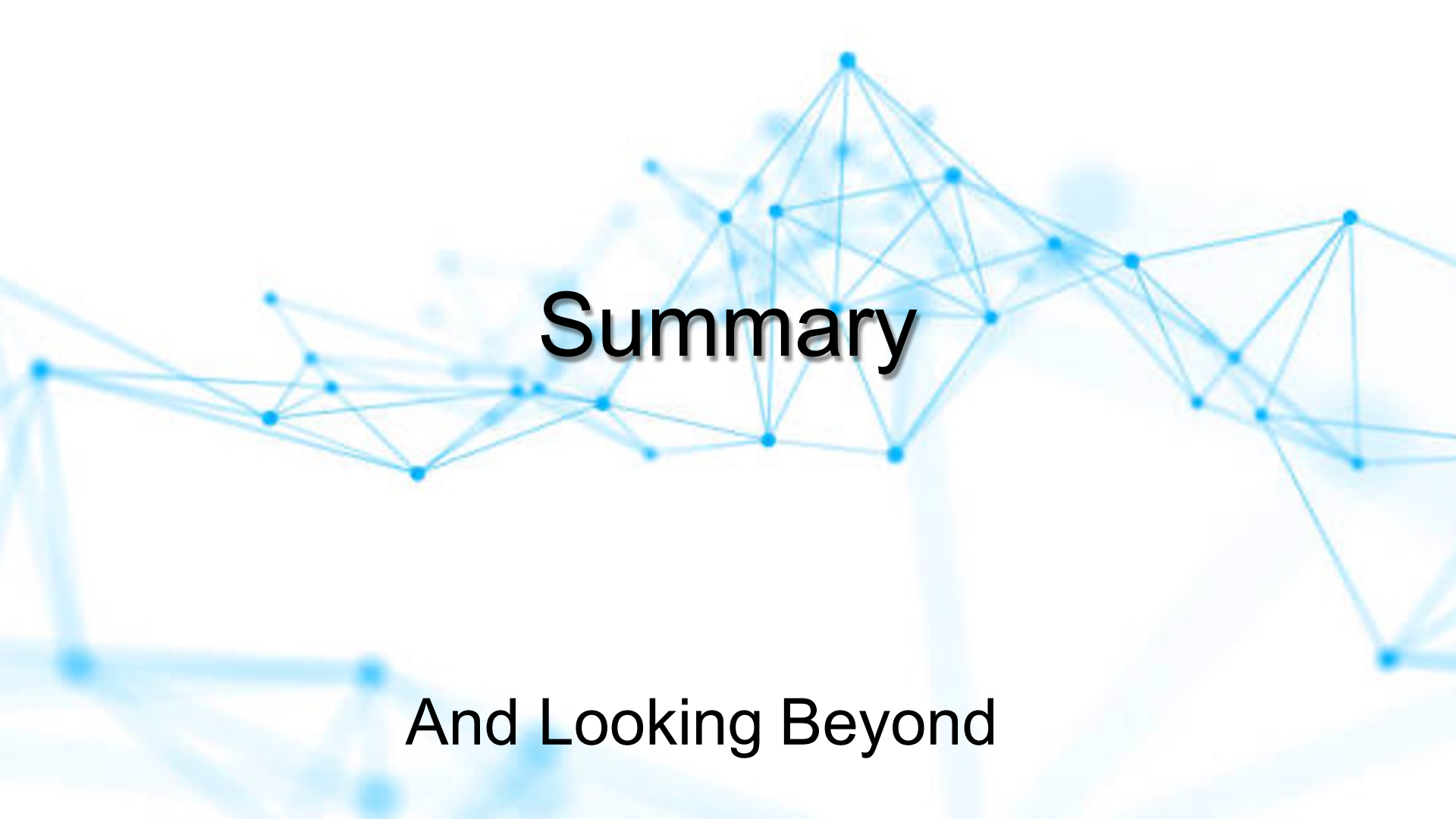
But:

reducer complexity $\downarrow$

-you control R
(specific tradeoffs
depend on data
and hardware)

Vector to Matrices

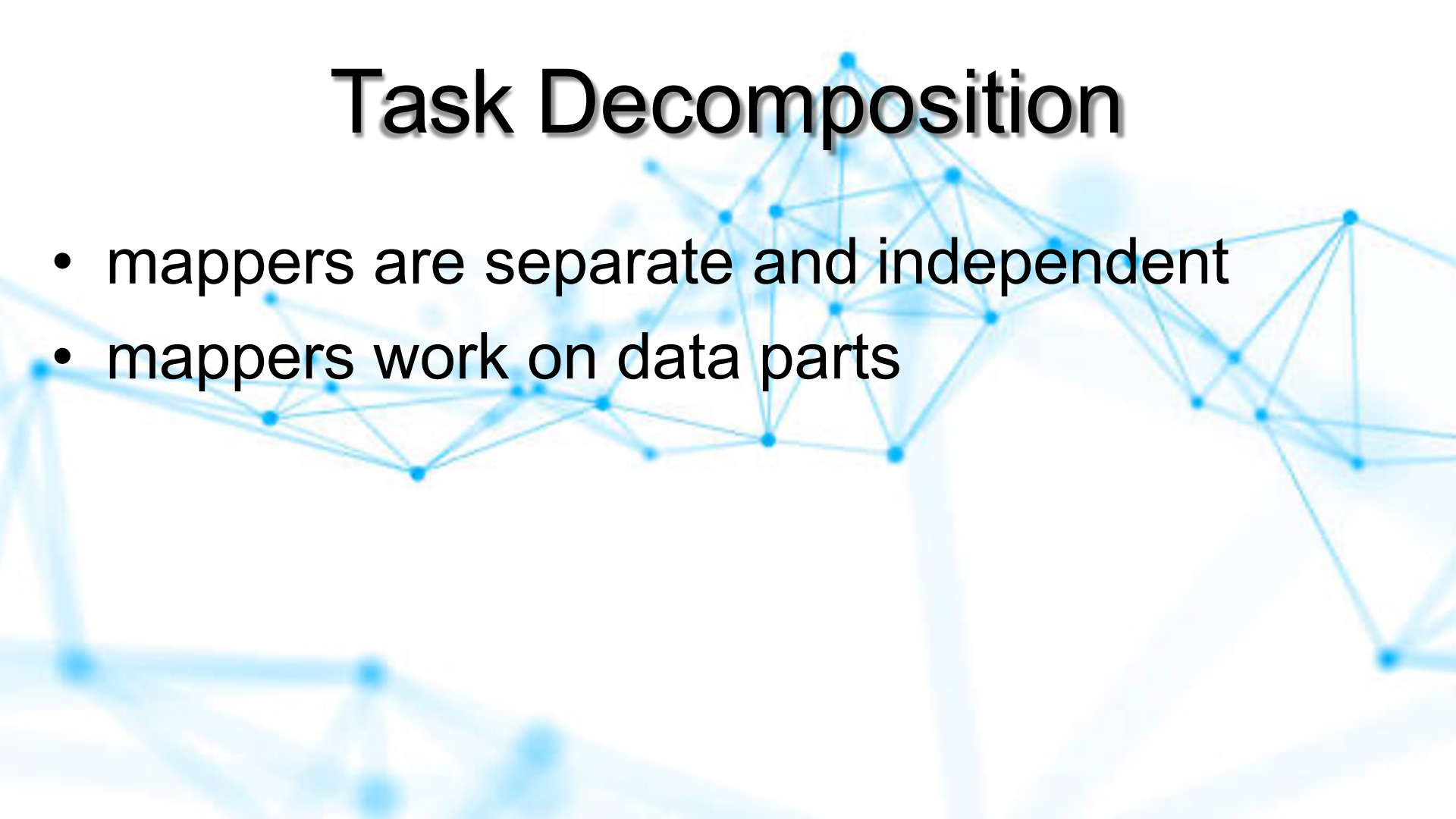
- Matrix multiplication needs row-index and col-index in the keys
- Matrix multiplication more pertinent to data analytic topics



Summary

And Looking Beyond

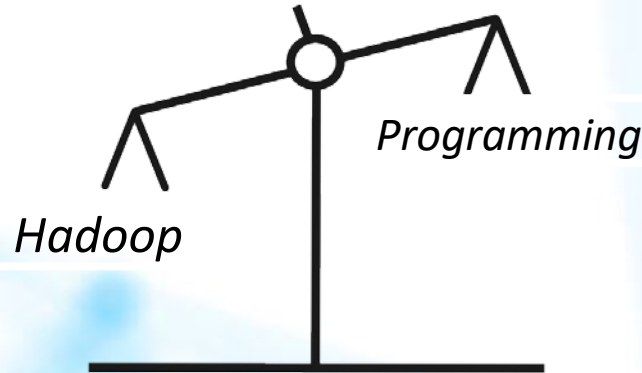
Task Decomposition



- mappers are separate and independent
- mappers work on data parts

Summary of Map/Reduce Design Considerations

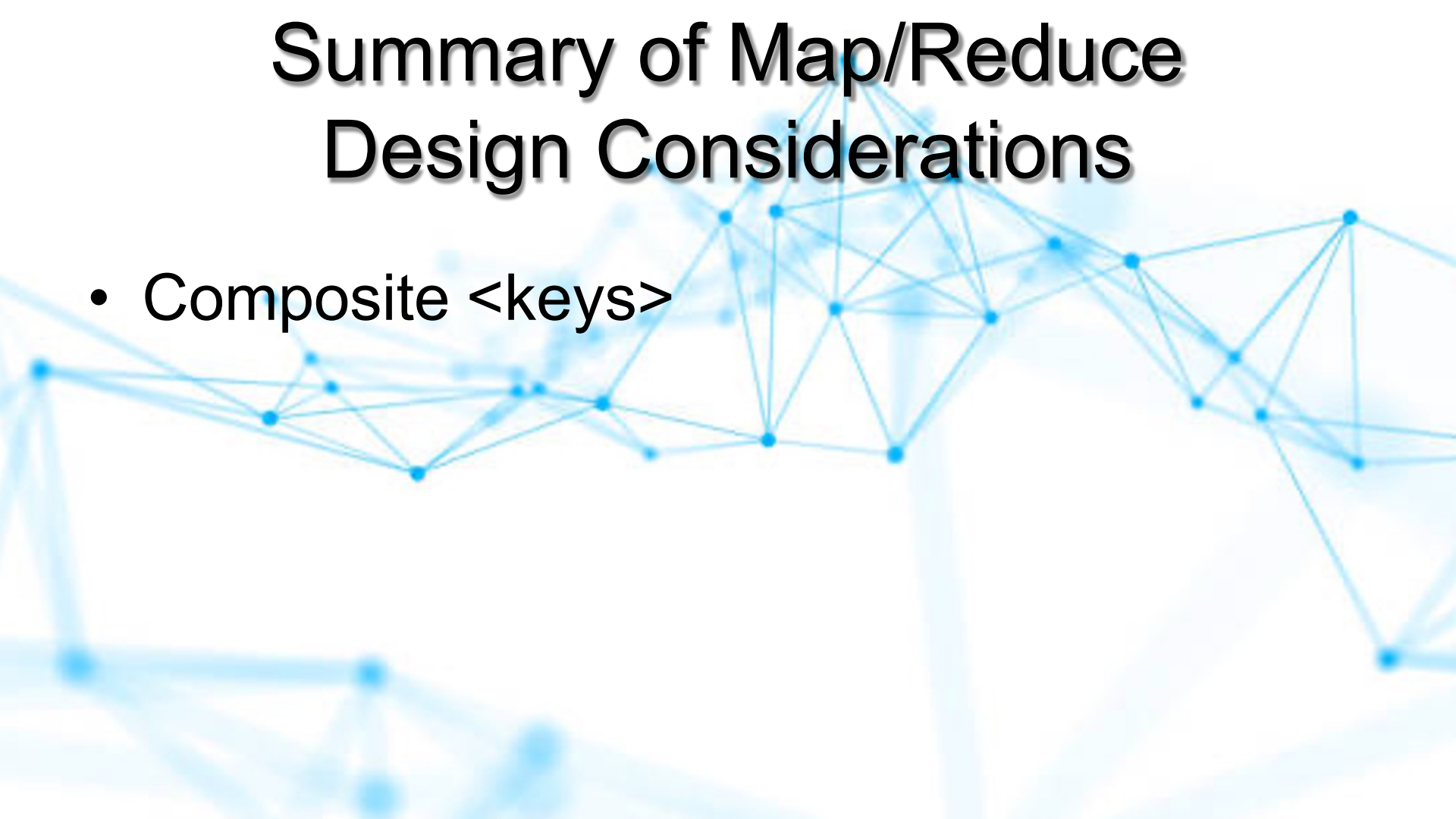
- $\langle \text{key}, \text{value} \rangle$ must enable correct output
- Let Hadoop do the hard work
- Trade-offs



Summary of Map/Reduce Design Considerations

- Common mappers:
 - Filter (subset data)
 - Identity (just pass data)
 - Splitter (as for counting)

Summary of Map/Reduce Design Considerations



- Composite <keys>

Summary of Map/Reduce Design Considerations

- Composite <keys>
- Extra info in <values>

Summary of Map/Reduce Design Considerations

- Composite <keys>
- Extra info in <values>
- Cascade Map/Reduce jobs

Summary of Map/Reduce Design Considerations

- Composite <keys>
- Extra info in <values>
- Cascade Map/Reduce jobs
- Bin keys into ranges

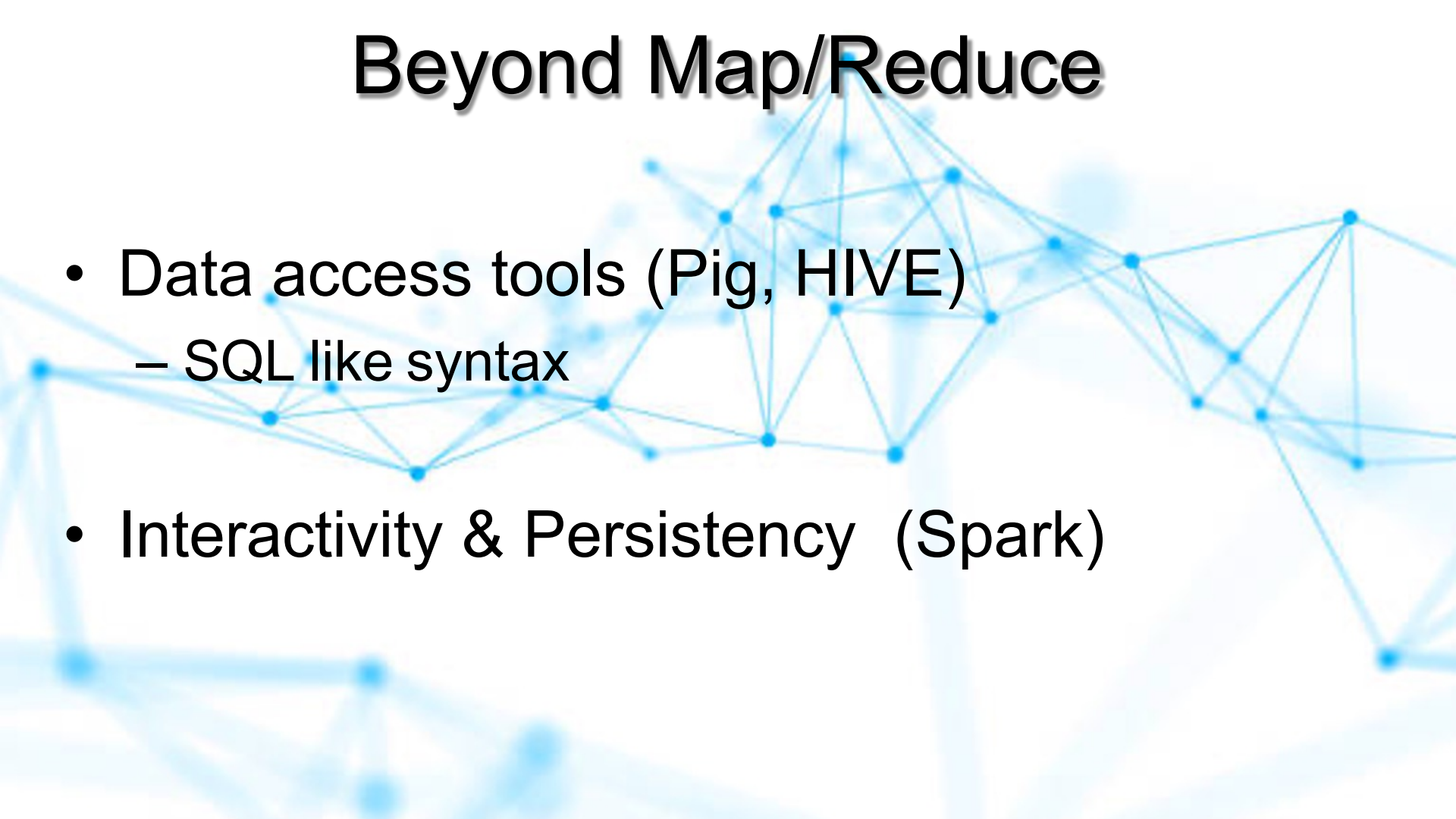
Summary of Map/Reduce Design Considerations

- Composite <keys>
- Extra info in <values>
- Cascade Map/Reduce jobs
- Bin keys into ranges
- Aggregate map output when possible
(combiner option)

Potential Limitations Map/Reduce

- Must fit <key, value> paradigm
- Map/Reduce data not persistent
- Requires programming/debugging
- Not interactive

Beyond Map/Reduce



- Data access tools (Pig, HIVE)
 - SQL like syntax
- Interactivity & Persistency (Spark)